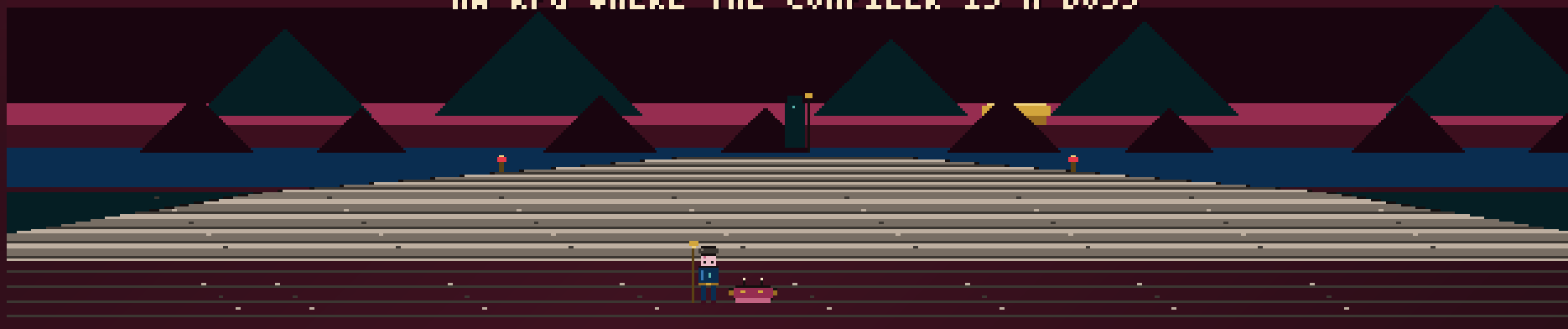




PLEDGE & CROWN

AN RPG WHERE THE COMPILER IS A BOSS



PRESS START

A PIXEL-ART RPG THAT TEACHES YOU RUST

The only way to act on the world is to write real, compilable Rust. Combat is code. Crafting is code. The Borrow Checker is a boss.

THE IDEA IN ONE BREATH

“**Pledge & Crown** is a pixel-art RPG where the only way to act on the world is to write real, compilable Rust. The 10-act campaign covers a complete, university-level Rust curriculum — variables to async to *unsafe* to FFI. The Borrow Checker is an NPC who guards a bridge and rejects your code with the **exact errors rustc would produce**. By the time a player finishes the campaign, they are a junior-to-mid Rust developer.

THE PROBLEM

Rust has the steepest learning cliff of any mainstream language.

Books & tutorials

The official Book, *Programming Rust*, Rustlings. High quality — low engagement. Most readers abandon mid-ownership chapter.

Programming games

CodeCombat, Mimo, Zachtronics. High engagement — but none teach Rust. They do JS/Python/Lua or fictional assembly.

There is no third option. We are the third option.

THE SOLUTION

You don't read about Rust. You wield it.

Every meaningful action in the world is a function you write. The engine compiles it in a sandbox, runs it, and the world responds.

Combat is code

Crafting is code

Town-building is code

Dialogue puzzles are code

Clean, `clippy`-idiomatic code hits harder. `.unwrap()` in the wrong place gets you hurt. The game has *opinions* — the same ones a senior reviewer would.



The Borrow Checker

A robed figure of glowing reference symbols who guards a bridge. He speaks in real rustc errors. You cannot brute-force past him.

The Cast Editor

CAST	TEST	COMMIT	ESC
<pre>// Defeat the goblin (HP 12). fn cast() → Spell { Spell { damage: 4 * 4, // 16 > 12 } }</pre>		GOAL Return a Spell whose damage exceeds the goblin's HP. COMPILER OUTPUT <pre>error[E0382]: borrow of moved value: `spell` --> cast.rs:6:18</pre> The errors you see are the real rustc errors — never paraphrased. Reading them is a skill we teach.	

- 1**
Hit a problem in the world
- 2**
Editor opens — write Rust
- 3**
Server compiles to WASM
- 4**
Sandbox runs your code
- 5**
The world reacts

THE WORLD — LIVE, IN-ENGINE

Hearthstone Village & beyond



Hearthstone Square — the **golden !** marks your next quest-giver (here, Ferris).

A designed town

Hub square + eight themed quarters — not procedural noise.



The south road — a hand-designed street grid through grass quarters, flowers & props.

Ten acts, ten zones

the Bridge, the Mage's Tower, the Forge, the Vault...

Locked palette & score

32-color "Heraldic Code" + original synthwave.

THE CAST

Every concept has a face. 57 NPCs, all original art.

Ferris guides you. The Borrow Checker tests you. The Trait Mage, the Quartermaster, the Tavern crew — each teaches a slice of Rust. (Actual in-game sprites, below.)



Ferris



Borrow Checker



Trait Mage



Oracle



Guildmaster



Quartermaster



Smith



Herald



Barkeep



Alchemist



Armorer



Loremaster



Auditor



Bellringer



Bondsmith



Bouncer



Cartographer



Cellarer



Chronicler



Conjurer



Cooper



Dockmaster



Drillmaster



Familiar



Forgewright



Ghostkeeper



Harbormaster



Heraldic Sage



Interpreter



Keymaster



Lanternkeeper



Lighthousekeeper



Linguist



Locksmith



Mixologist



Pilgrim



Porter



Reckoner



Recruiter



Riveter



Sharekeeper



Sifter



Signaler



Smelter



Strongbox



Surveyor



Swapwarden



Tabkeeper



Tallywright



Tideforecaster



Tidewatch



Tinker



Twin



Vaultwright



Vexis



Wandwright



Warden

A full Rust education, in ten acts.

25–35 hours · every concept in the Rust Book through Chapter 20 · reachable through normal play.

I	Hearthstone Village	variables, types, control flow, loops	II	The Borrow Checker's Bridge	ownership, borrowing, lifetimes, slices
III	Guildhall Quarter	structs, enums, pattern matching, impl	IV	Trait Mage's Tower	traits, generics, dyn, associated types
V	Tavern of Tribulations	Result, Option, ?, custom errors	VI	Iterator Forge	collections, iterators, closures
VII	Concurrent Coast	threads, Arc<Mutex>, channels, async	VIII	Vault of Pointers	Box, Rc, RefCell, Weak
IX	Forbidden Library	unsafe, FFI, macros, Drop/Send/Sync	X	Throne of the Compiler	performance, no_std, the ecosystem

The player leaves as a **Rust developer**. That's the north star.

It eats its own dog food.

A Rust-teaching game, built entirely in Rust — the codebase *is* a teaching artifact.

Engine

Bevy ECS, native + WebGL2. `bevy_egui` + `egui_code_editor` drive the in-game IDE with real Rust syntax highlighting.

Player code

Source → server-side cargo compile → WASM. Runs in a fuel-limited sandbox — **never the host process**. The server owns the manifest.

Everywhere

One Rust codebase ships to desktop (native), browser (wasm), and mobile (Tauri 2.0, Android bundle builds today).

Bevy

`bevy_egui``egui_code_editor`

Axum compile API

WASM sandbox

`wasm-bindgen`

Tauri 2.0

No unsafe in shipped game code

This is not a concept deck. It runs.

- ❖ **57 missions** across Acts 1–8, fully playable, strict-linear curriculum gating
- ❖ **57 NPCs**, every one with original sprite art
- ❖ Builds & runs **native + in-browser (WebGL2)**
- ❖ Original synthwave score + sound effects
- ❖ Android app bundle builds end-to-end (Tauri)

In progress

Final art-fidelity pass · Steam page · honest “compiled-and-ran” grading flipped to prod default.

Next on the roadmap

Acts 9–10 · the post-game **Temple** (a HolyC roguelike, kept as a surprise reveal) · B2B “team progress” dashboard.

Built solo, in <2 months, on a self-hosted toolchain — tested behind a full local CI gate (fmt + clippy -D warnings + ~100 grader tests + byte-parity contract suite).

Rust crossed the inflection point.

- ❖ In the **Linux** and **Windows** kernels; powering AWS Firecracker, Cloudflare, Discord.
- ❖ The Rust Survey's **#1 barrier, every year: the learning curve.**
- ❖ Indie edutainment distribution (itch.io, Steam) is healthier than ever for niches.
- ❖ Gen-AI art + LLM tutoring make a solo studio economically viable.

PRIMARY

Pro devs & CS students, 22–40, who tried Rust once and bounced. Spending power; will pay \$25–\$40 to finally get it.

SECONDARY

University CS courses — departmental licensing.

TERTIARY — THE GROWTH ENGINE

Engineering teams onboarding Rust. B2B at **\$50/seat**, where adoption is fastest.

BUSINESS MODEL

Free demo → paid game → B2B.

PHASE 0 · DEMO

Free

Acts 1–2 on itch.io. Top-of-funnel, email capture, validation.

PHASE 1 · EARLY ACCESS

\$14.99

Acts 1–4, itch.io. First dollars; Steam wishlist warms.

PHASE 2 · STEAM

\$24.99

Acts 1–7 + soundtrack DLC. Mainstream discoverability.

PHASE 3 · FULL + B2B

\$39.99

All 10 acts + Temple. Deluxe ed. + team licensing \$50/seat.

~\$110K

projected Year-1 net if phases land near target (conservative \$40K; stretch \$250K with a major creator feature). *Projection, not a guarantee.*

WHAT WE WILL NEVER DO

No microtransactions

No NFTs

No energy timers

No paywalled hints



A finished, playable Rust RPG — looking for the right partner to ship it.

Publishing / distribution

Funding the Acts 9–10 + Temple push

B2B / education pilots

Playtesters & feedback

Matt Gates · Chico, CA · github.com/suhteevah